Memorando

Aplicações Móveis, 2026/2027

De: Emanuel Carneiro dos Santos
Nº de Matrícula: 20230429
Assunto: Introdução ao Android com Kotlin
Data: 08/11/2026

1. Introdução

O presente laboratório, segunda parte do Laboratório 2, tem como principal objectivo a introdução ao desenvolvimento de aplicações Android com a linguagem **Kotlin** e o toolkit de interfaces **Jetpack Compose**, retomando os temas da Parte 1 (estrutura do projecto, execução, depuração e navegação entre actividades), agora sem layouts em XML.

O **Kotlin** é a linguagem recomendada pela Google para Android: tem sintaxe concisa, segurança de valor nulo integrada no sistema de tipos, funções de ordem superior e lambdas, e classes de dados. No **Jetpack Compose** a interface é declarada por funções anotadas com `@Composable`, que o runtime volta a executar (recomposição) sempre que o estado que lêem muda. A actividade passa a estender `ComponentActivity` e a definir a interface com `setContent { }`, em vez de `AppCompatActivity` com `setContentView(R.layout.…)`. A construção continua a cargo do Gradle (Kotlin DSL), agora com o plugin do compilador Compose.

São quatro os exercícios abordados: o **Exercício I** constrói a aplicação *Hello Android* com duas actividades, percorrendo a criação do projecto, a execução, o Logcat e o Toast, e a navegação por `Intent`; o **Exercício II** introduz o Kotlin no processamento da mensagem recebida (`val`/`var`, `when`, segurança nula, a classe de dados `PalavraBundle` e funções com lambdas); o **Exercício III** trata a depuração com níveis de log, um erro intencional, breakpoints e o debugger; o **Exercício IV** constrói uma aplicação de lista de tarefas com `LazyColumn` e a classe de dados `Task`. A secção de desafios apresenta uma panorâmica do software concebido: o desenho das actividades e os detalhes de implementação das duas aplicações.

Quanto às restrições, usou-se o Android Studio Rabbit 2 | 2026.2.2 Patch 1, o modelo «Empty Activity» (Compose) e o SDK mínimo API 24. Os testes foram feitos num dispositivo físico Samsung SM-A105G (Android 11, API 30), ligado por USB. A segunda actividade foi criada com o modelo Compose em vez do «Empty Views Activity» indicado, e corrigiram-se inconsistências dos exemplos do enunciado (`List` sem tipo, propriedades `count`/`averageLength` inexistentes e o duplo nome `gerarSumario`/`gerarResumo`, tendo-se adoptado `gerarResumo`).

OBS: As respostas às perguntas do laboratório encontram-se no final de cada subsecção que contém perguntas.

2. Experiências Realizadas

Descrevem-se, de seguida, os Exercícios I a IV do enunciado, pela ordem em que foram realizados.

2.1. Exercício I - Hello Android

O objectivo deste exercício é construir uma aplicação com duas actividades: a primeira recolhe uma mensagem numa caixa de texto e, ao pressionar «Enviar», lança a segunda, que apresenta essa mensagem.

2.1.1. Criar o projecto Android e entender a sua estrutura

O projecto foi criado a partir do ecrã inicial do Android Studio (Figura 1) com o modelo **Empty Activity** (Figura 2). Ao contrário da Parte 1, este é precisamente o modelo pretendido: gera uma actividade em Kotlin com Jetpack Compose, pelo que o assistente não pergunta a linguagem.

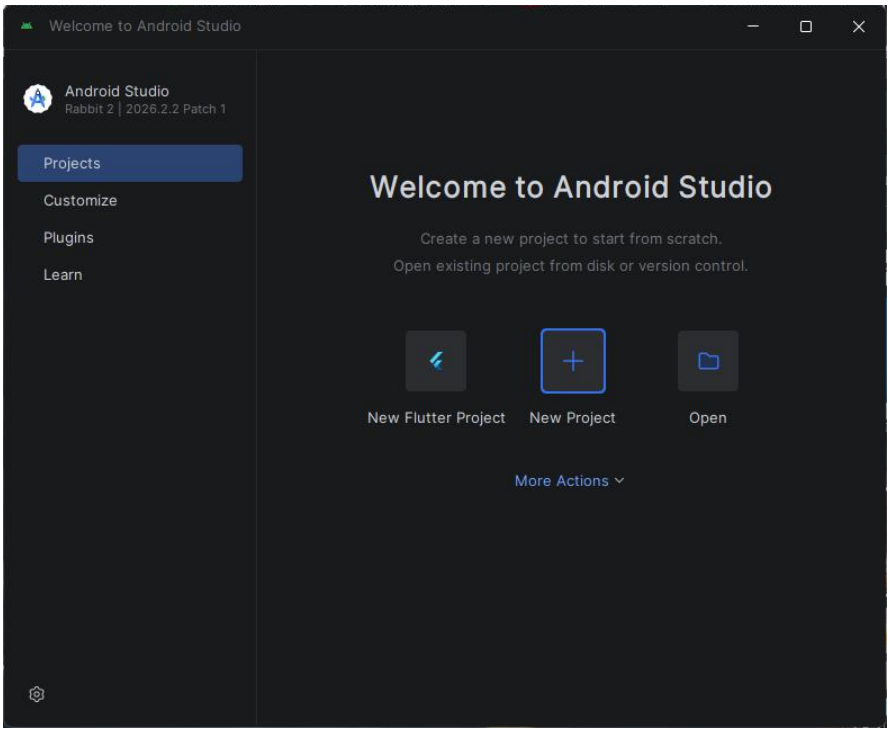


Figura 1 **–** Ecrã de boas-vindas do Android Studio (Rabbit 2 | 2026.2.2 Patch 1), ponto de partida para a criação do projecto

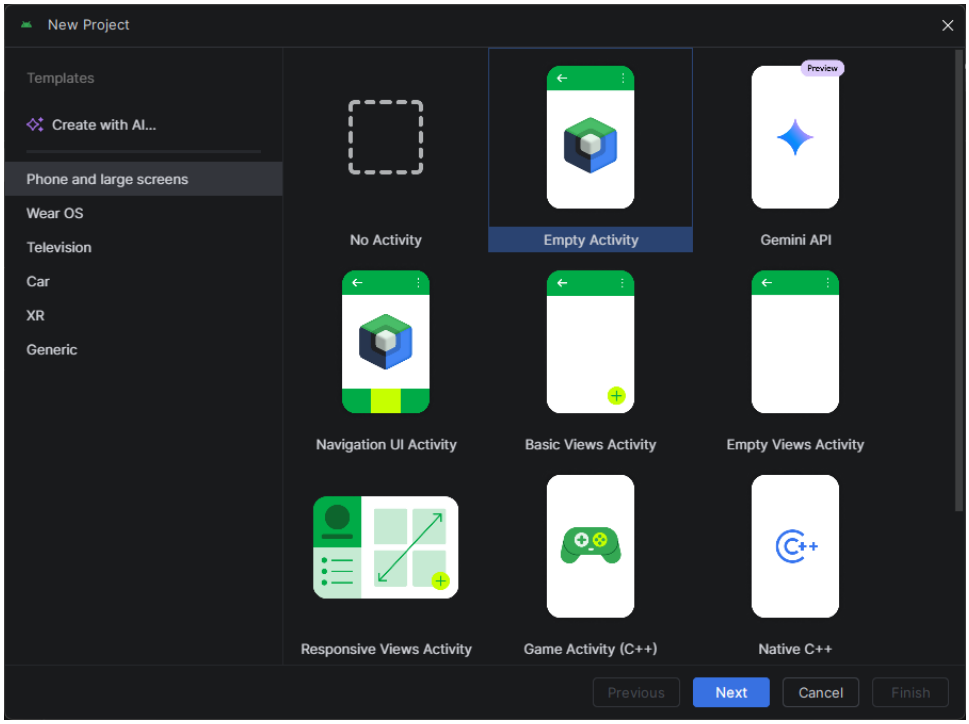


Figura 2 **–** Selecção do modelo Empty Activity, que gera uma actividade Jetpack Compose em Kotlin

Na configuração (Figura 3) definiram-se o nome *HelloAndroid*, o pacote `ao.co.isptec.aplm.helloandroid` e o SDK mínimo API 24 (Android 7.0). O nome é relevante porque dele deriva o tema gerado, `HelloAndroidTheme`, usado no código do enunciado. A Figura 4 mostra a árvore do projecto na vista **Project** e a `MainActivity` gerada.

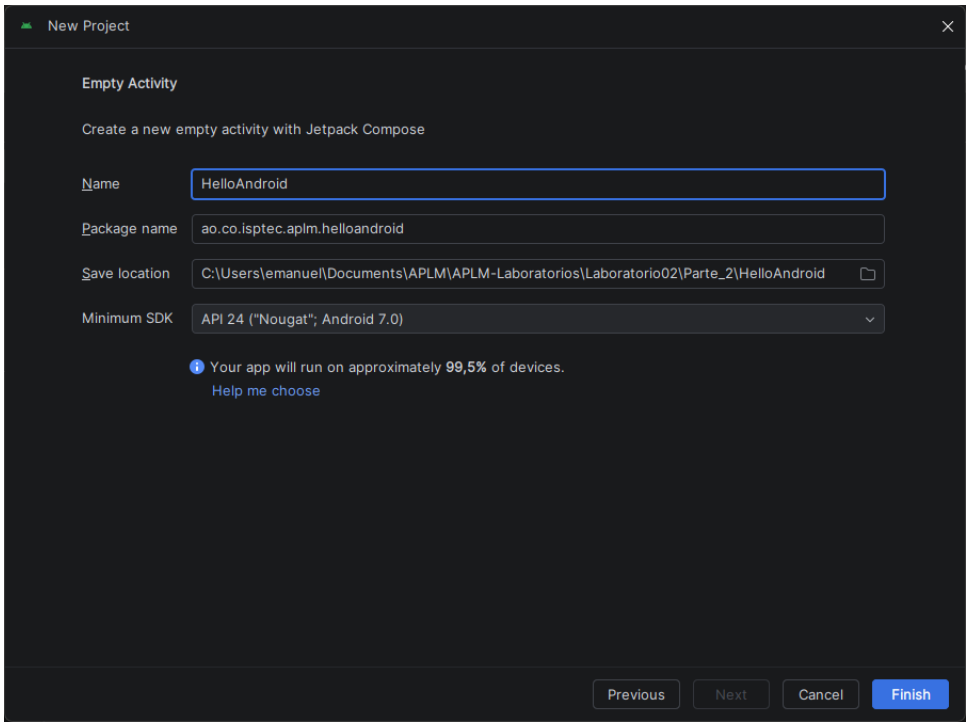


Figura 3 **–** Configuração do projecto HelloAndroid: pacote ao.co.isptec.aplm.helloandroid e SDK mínimo API 24

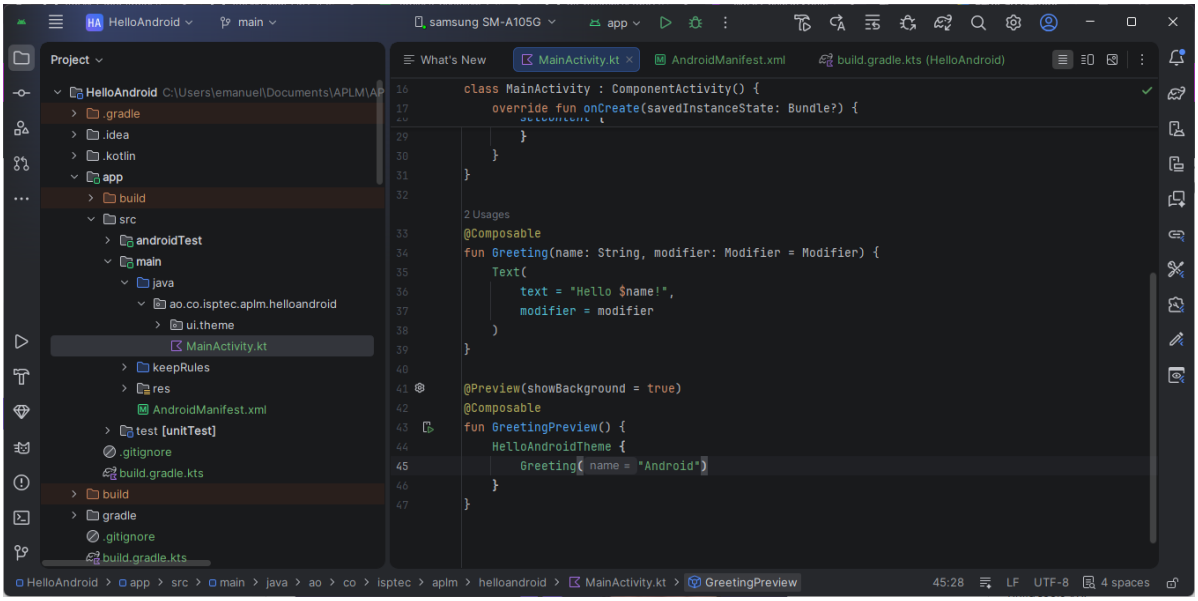


Figura 4 **–** Árvore do projecto na vista Project e MainActivity.kt gerada pelo modelo, com as funções Greeting e GreetingPreview

1) Qual é a função dos ficheiros `MainActivity.kt`, `AndroidManifest.xml` e `build.gradle.kts` (projecto e módulo)?

R:

`MainActivity.kt` (app/src/main/java/ao/co/isptec/aplm/helloandroid) - actividade de entrada da aplicação; estende `ComponentActivity` e, em `onCreate()`, define a interface com `setContent { }`, onde se chamam as funções `@Composable` envolvidas no tema `HelloAndroidTheme`.

`AndroidManifest.xml` (app/src/main) - esqueleto da aplicação: declara as actividades (com `android:exported` e o `intent-filter` MAIN/LAUNCHER que define a actividade inicial), o ícone, o rótulo e o tema. O sistema lê-o antes de executar qualquer código.

`build.gradle.kts` (projecto) - configuração comum a todos os módulos; declara os plugins (aplicação Android, Kotlin e compilador Compose) sem os aplicar (`apply false`), com versões do catálogo `libs.versions.toml`.

`app/build.gradle.kts` (módulo) - aplica esses plugins e define `namespace`, `compileSdk`, `minSdk` 24, `targetSdk`, os tipos de build, a activação do Compose (`buildFeatures { compose = true }`) e as dependências (BOM do Compose, Material 3, `activity-compose`, entre outras).

2.1.2. Executar a aplicação

No **Device Manager** configurou-se um dispositivo virtual Pixel 6 com a imagem Google APIs API 34 (x86_64), conforme o enunciado (Figura 5). A imagem do sistema exige uma descarga de 1,5 GB; tal como na Parte 1, a aplicação foi executada no dispositivo físico Samsung SM-A105G, com a depuração USB activa. A Figura 6 mostra a primeira execução, espelhada no painel *Running Devices*.

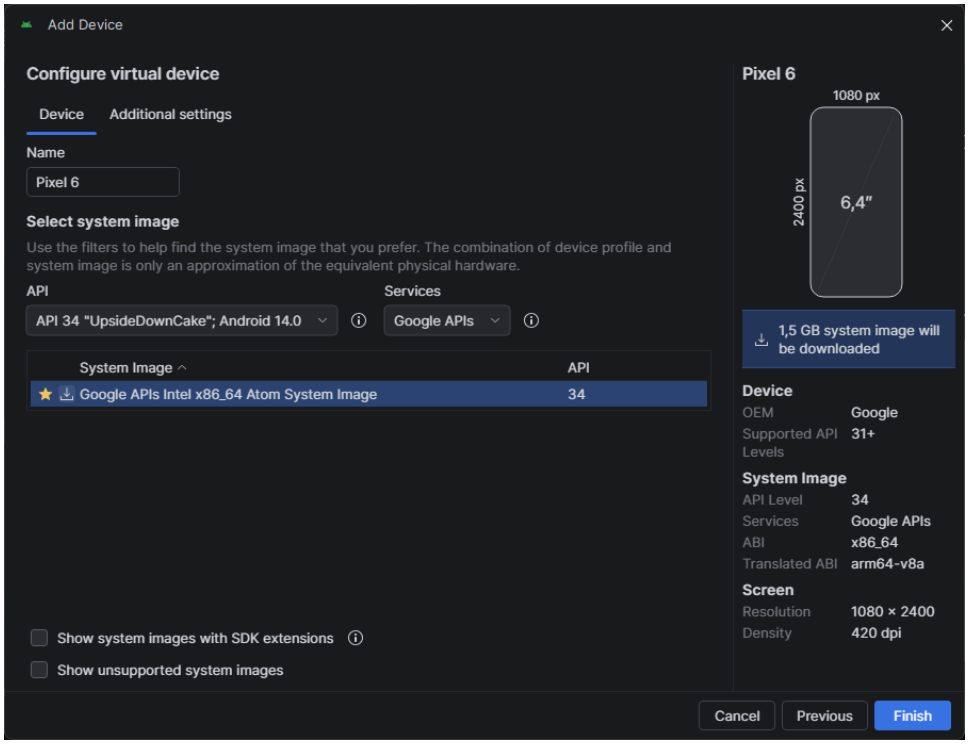


Figura 5 **–** Configuração do dispositivo virtual Pixel 6 com a imagem Google APIs API 34 (x86_64) no Device Manager

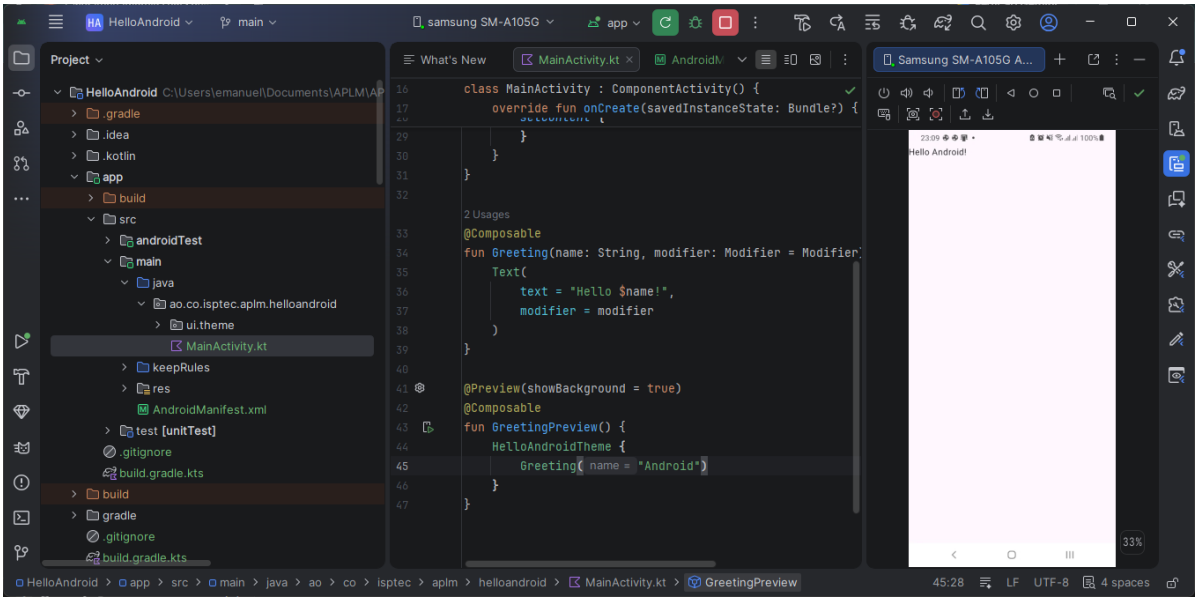


Figura 6 **–** Primeira execução da aplicação no Samsung SM-A105G, espelhado no painel Running Devices («Hello Android!»)

2.1.3. Logging e feedback

No método `onCreate()` da `MainActivity` acrescentou-se a seguinte instrução, com o respectivo `import android.util.Log`:

```
Log.d("MainActivity", "Aplicação iniciada com sucesso.")
```

Ao executar, a mensagem surge no Logcat com o nível D e a tag `MainActivity`, usando o filtro `package:mine tag:MainActivity` (Figura 7).

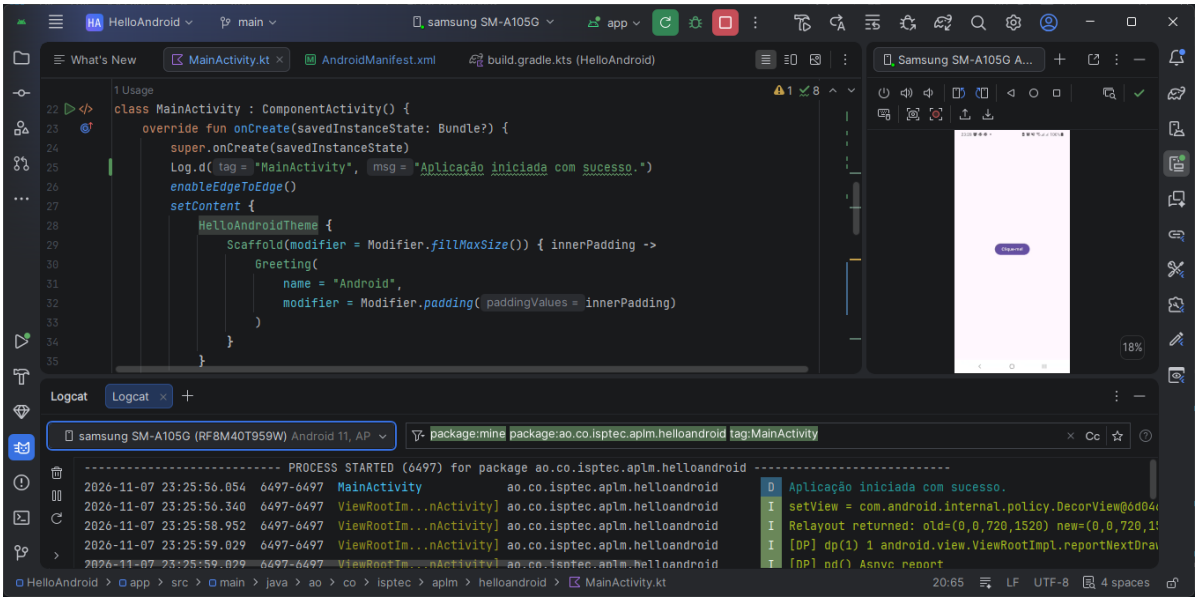
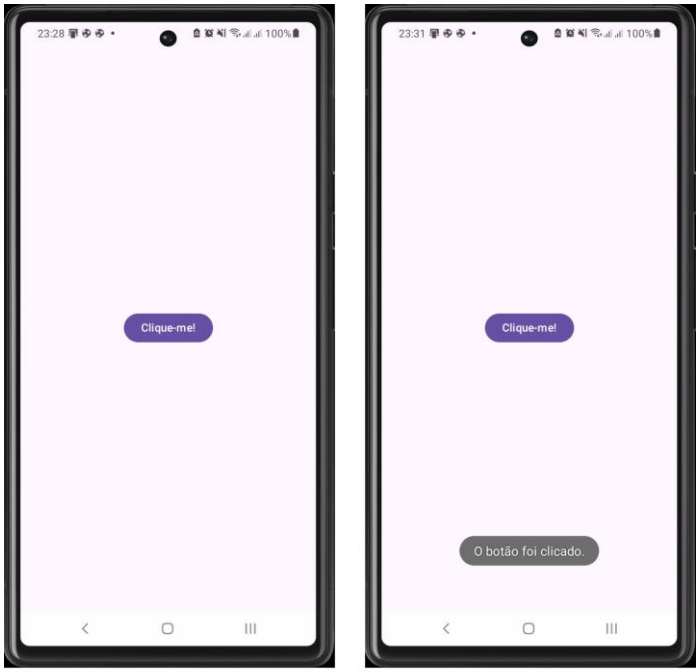


Figura 7 **–** Mensagem «Aplicação iniciada com sucesso.» no Logcat, filtrada pelo pacote da aplicação e pela tag MainActivity

Em seguida, o conteúdo de `Greeting()` foi substituído por um botão centrado numa `Box`, que mostra um Toast e regista um log a cada clique:

```
val context = LocalContext.current
Box(modifier = modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
    Button(onClick = {
        Toast.makeText(context, "O botão foi clicado.", Toast.LENGTH_SHORT).show()
        Log.d("MainActivity", "Botão clicado: Toast apresentado.")
    }) {
        Text("Clique-me!")
    }
}
```

As Figuras 8 e 9 confirmam o funcionamento: o Toast é apresentado no dispositivo e a mensagem do clique surge no Logcat.



(a) botão «Clique-me!» (b) Toast após o clique

Figura 8 **–** Botão criado em Compose: (a) ecrã com o botão «Clique-me!»; (b) Toast «O botão foi clicado.» apresentado após o clique

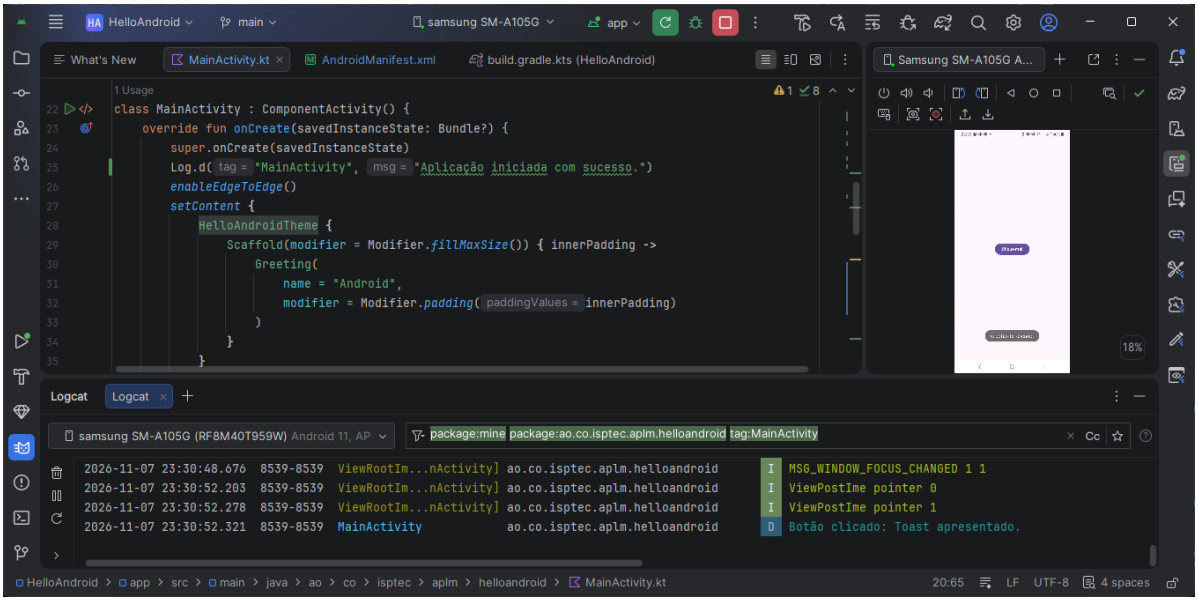


Figura 9 **–** Log «Botão clicado: Toast apresentado.» registado no Logcat após o clique

2) Onde posso configurar como os logs são filtrados no Logcat?

R:

Na barra de consulta, no topo da janela do Logcat, onde se escrevem expressões `chave:valor`: `package:` (por exemplo `package:mine`), `tag:`, `level:`, `message:`, `process:`, `age:` e `is:crash`. As chaves podem combinar-se; o prefixo `-` exclui e o sufixo `~` usa expressões regulares. A barra sugere chaves e valores e permite guardar consultas como favoritas (estrela); o menu à esquerda escolhe o dispositivo. O filtro por omissão de cada nova janela define-se em Settings > Tools > Logcat.

2.1.4. Criar uma segunda actividade

O componente `Greeting` foi substituído pelo composable `MessageScreen`: uma `Column` com um `OutlinedTextField` e um botão «Enviar». O texto é guardado num estado criado com `remember { mutableStateOf("") }` e, ao clicar, o botão mostra um Toast, regista a mensagem com `Log.d` e lança a segunda actividade com uma `Intent` explícita. A chave do extra foi definida como constante de nível superior, partilhada pelas duas actividades:

```
const val EXTRA_MESSAGE = "EXTRA_MESSAGE"

var text by remember { mutableStateOf("") }
...
Button(onClick = {
    Toast.makeText(context, "Botão clicado!", Toast.LENGTH_SHORT).show()
    Log.d(TAG, "Utilizador digitou a mensagem: $text")
    val intent = Intent(context, DisplayMessageActivity::class.java).apply {
        putExtra(EXTRA_MESSAGE, text)
    }
    context.startActivity(intent)
}) { Text("Enviar") }
```

A `DisplayMessageActivity` foi criada com New > Activity > **Empty Activity** (Figura 10). O enunciado indica «Empty Views Activity», mas num projecto Compose esse modelo gera um layout XML e uma classe `AppCompatActivity` que teria depois de ser adaptada; o modelo Compose já gera a classe a estender `ComponentActivity`, com `setContent`, e regista-a no manifesto (Figura 11). A actividade recupera a mensagem e apresenta-a num `Text`:

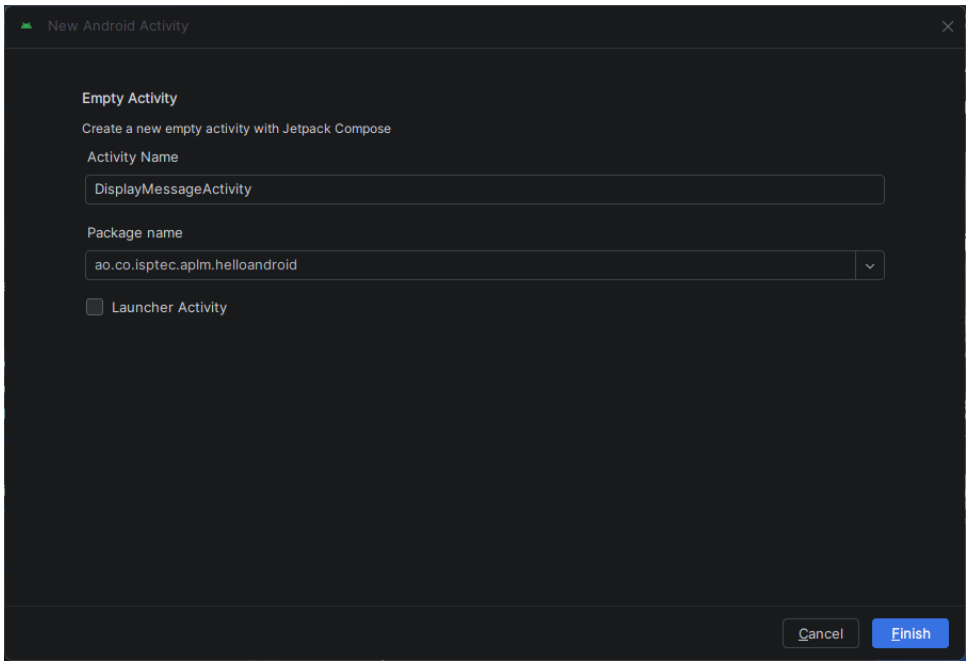


Figura 10 **–** Criação da DisplayMessageActivity com o modelo Empty Activity (Jetpack Compose), no pacote ao.co.isptec.aplm.helloandroid

```
val message = intent.getStringExtra(EXTRA_MESSAGE) ?: "Nenhuma mensagem recebida."
setContent {
    HelloAndroidTheme {
        Box(contentAlignment = Alignment.Center, modifier = Modifier.fillMaxSize()) {
            Text(text = message, fontSize = 24.sp)
        }
    }
}
```

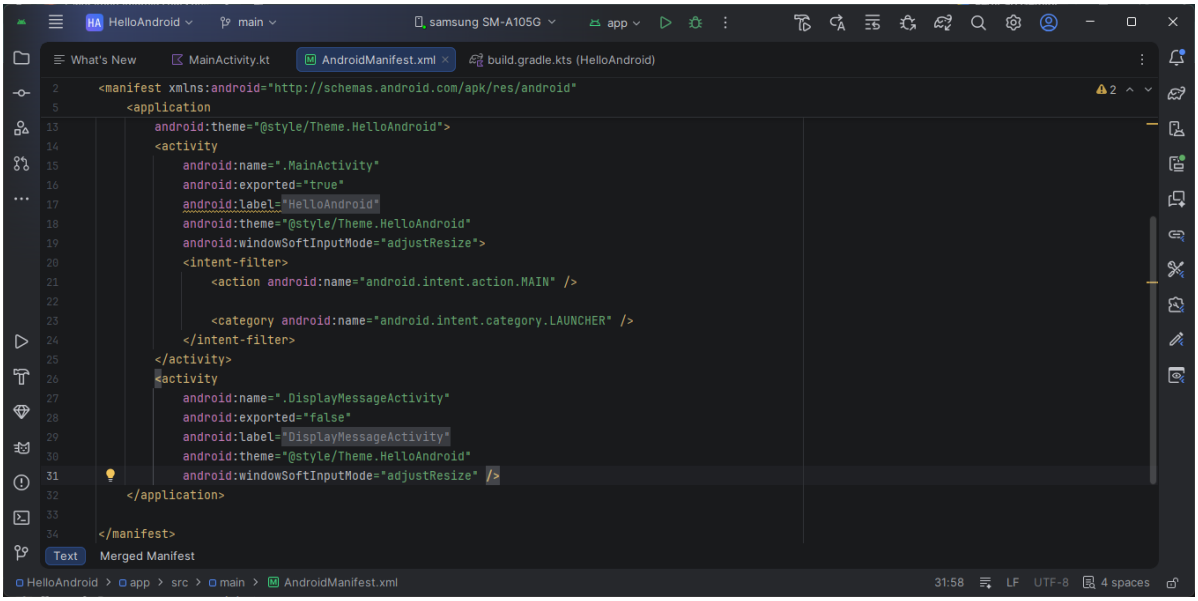
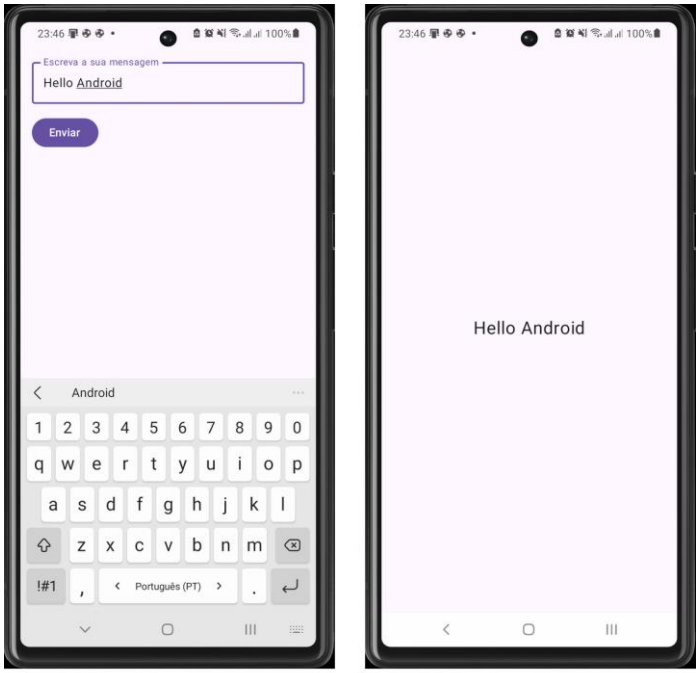


Figura 11 **–** AndroidManifest.xml com a MainActivity (launcher, exported="true") e a DisplayMessageActivity (exported="false") declaradas em <application>



(a) MainActivity (b) DisplayMessageActivity

Figura 12 **–** Navegação entre actividades: (a) mensagem «Hello Android» escrita na MainActivity; (b) a mesma mensagem apresentada pela DisplayMessageActivity

Durante este passo, o editor assinalou erros como «Cannot access class androidx.compose.runtime.internal.ComposableFunction0» e «Argument type mismatch» nas lambdas composable, embora o projecto compilasse e executasse correctamente. Estes tipos internos são gerados pelo plugin do compilador Compose, que o Gradle aplica no build; os erros vêm apenas da análise do editor, desalinhada com esse plugin (caches desactualizadas após o primeiro sync ou versões diferentes do plugin Kotlin do IDE). Corrigem-se sincronizando o projecto (Sync Project with Gradle Files) ou com File > Invalidate Caches.

3) Qual o papel do objecto `Intent` nesta aplicação?

R:

É a mensagem que pede ao sistema para iniciar a segunda actividade. Trata-se de uma intent explícita: indica o `Context` de origem e a classe de destino (`DisplayMessageActivity::class.java`) e é entregue a `startActivity()`. Também transporta os dados entre actividades: a mensagem vai como extra (`putExtra(EXTRA_MESSAGE, text)`) e é lida no destino com `intent.getStringExtra(EXTRA_MESSAGE)`.

4) Qual o papel de `remember` e `mutableStateOf`?

R:

`mutableStateOf("")` cria um `MutableState` observável: quando o seu valor muda, o Compose recompõe os composables que o lêem, actualizando o `OutlinedTextField`. `remember` guarda esse objecto entre recomposições; sem ele, o estado seria recriado vazio a cada execução da função e o texto perder-se-ia. A delegação `by` permite ler e escrever `text` como uma variável comum. O estado não sobrevive à recriação da actividade (por exemplo, numa rotação); para isso usa-se `rememberSaveable`.

5) Porquê precisamos do `LocalContext.current` num composable para iniciar uma actividade?

R:

Porque `Intent(context, …)`, `startActivity()` e `Toast.makeText()` precisam de um `Context`, e um composable é uma função, não um objecto `Context` com acesso a `this`. `LocalContext` é um `CompositionLocal` fornecido por `setContent` que disponibiliza o `Context` da actividade a qualquer composable da árvore, sem ser preciso passá-lo de parâmetro em parâmetro.

2.2. Exercício II - Introdução ao Kotlin

Neste exercício a `DisplayMessageActivity` passou a processar a mensagem recebida, introduzindo progressivamente as construções do Kotlin.

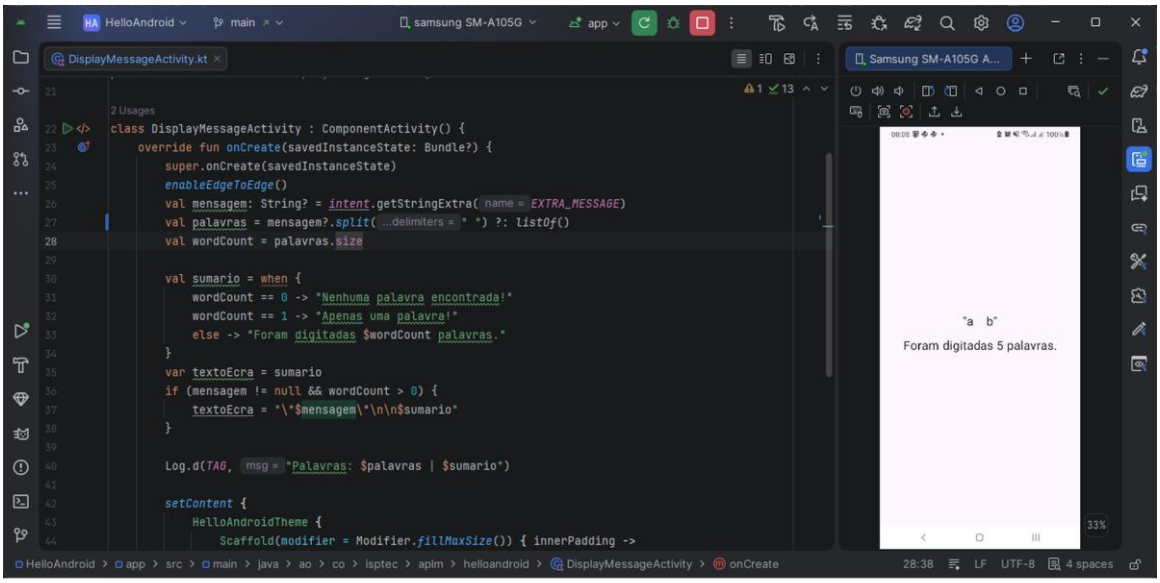
2.2.1. Variáveis, controlo de fluxo e segurança de valor nulo

A mensagem é lida como `String?`, dividida em palavras e classificada com `when` como expressão; uma `var` é reatribuída num `if` consoante o número de palavras:

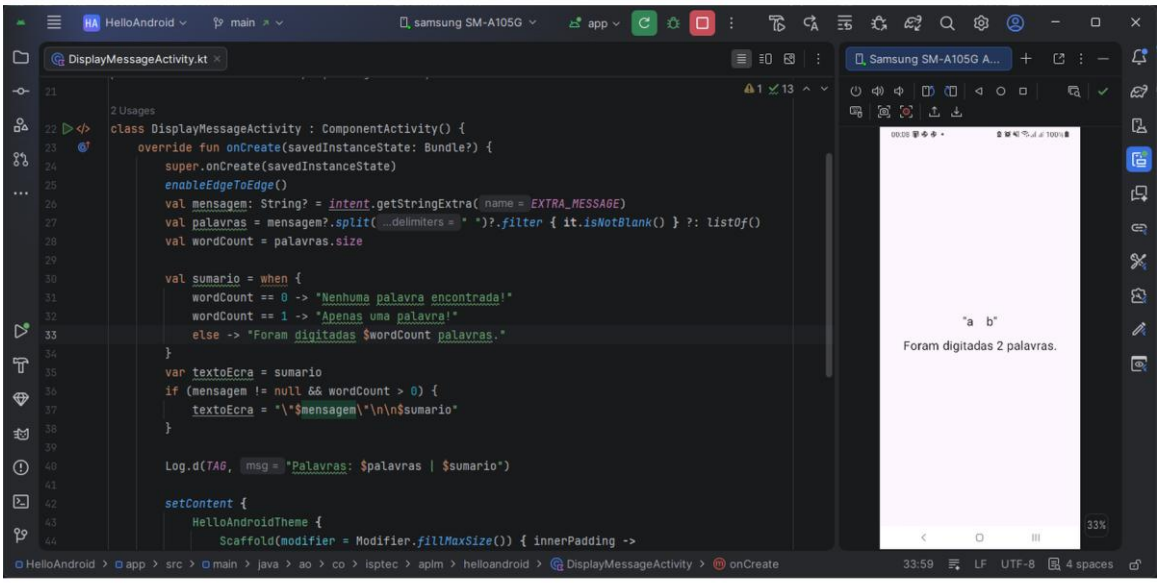
```
val mensagem: String? = intent.getStringExtra(EXTRA_MESSAGE)
val palavras = mensagem?.split(" ")?.filter { it.isNotBlank() } ?: listOf()
val wordCount = palavras.size

val sumario = when {
    wordCount == 0 -> "Nenhuma palavra encontrada!"
    wordCount == 1 -> "Apenas uma palavra!"
    else -> "Foram digitadas $wordCount palavras."
}
var textoEcra = sumario
if (mensagem != null && wordCount > 0) {   // smart cast: mensagem é String
    textoEcra = "\"$mensagem\"\n\n$sumario"
}
```

O código do enunciado (`message?.split(" ") ?: listOf()`) conta mal as palavras: `"".split(" ")` devolve `[""]` (uma «palavra» vazia) e cada espaço repetido gera mais um elemento vazio. A Figura 13 mostra o mesmo texto com vários espaços, contado como 5 palavras sem o filtro e como 2 com `filter { it.isNotBlank() }`. A Figura 14 apresenta os três ramos do `when`.

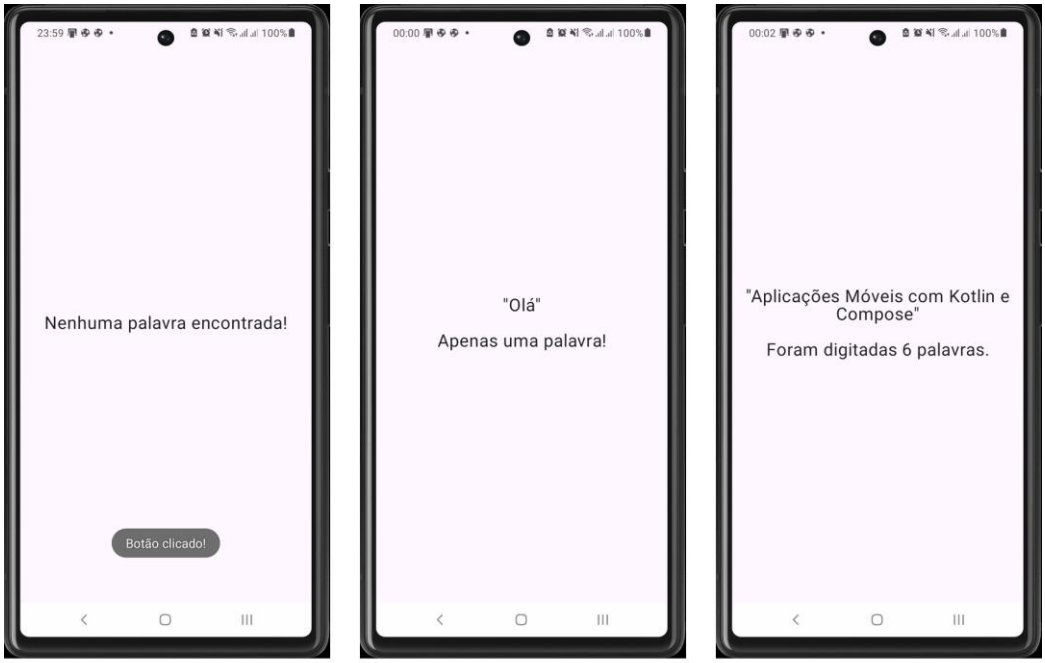


(a) sem filter: 5 palavras



(b) com filter { it.isNotBlank() }: 2 palavras

Figura 13 **–** Efeito do filtro de elementos vazios: (a) sem filter, o texto «a» e «b» separados por vários espaços conta 5 palavras; (b) com filter { it.isNotBlank() }, conta 2



(a) mensagem vazia (b) «Olá» (c) frase de 6 palavras

Figura 14 **–** Resposta do when consoante o número de palavras: (a) mensagem vazia; (b) uma palavra; (c) seis palavras

6) Como seria essa mesma lógica em Java? O que significa `?.split()`? O que faz o operador `?:`?

R:

Em Java a verificação de nulo e a selecção têm de ser explícitas:

```
String message = getIntent().getStringExtra("EXTRA_MESSAGE");
List<String> words = new ArrayList<>();
if (message != null) {
    for (String w : message.split(" ")) {
        if (!w.trim().isEmpty()) words.add(w);
    }
}
int wordCount = words.size();
String summary;
if (wordCount == 0) summary = "Nenhuma palavra encontrada!";
else if (wordCount == 1) summary = "Apenas uma palavra!";
else summary = "Foram digitadas " + wordCount + " palavras.";
```

`?.` é a chamada segura: `split` só é invocado se `mensagem` não for `null`; caso contrário a expressão inteira vale `null`, sem `NullPointerException`. `?:` é o operador Elvis: devolve o operando da esquerda se não for `null` e, senão, o da direita (`listOf()`), garantindo uma `List<String>` não nula.

2.2.2. Classes de dados e operações de lista

Criou-se o ficheiro `PalavraBundle.kt` com uma `data class`, corrigindo o tipo da propriedade para `List<String>`. As estatísticas são propriedades computadas; o comprimento médio de uma lista vazia daria `NaN` em `average()`, pelo que esse caso devolve 0.0:

```
data class PalavraBundle(val palavras: List<String>) {
    val contar: Int
        get() = palavras.size
    val comprimentoMedio: Double
        get() = if (palavras.isEmpty()) 0.0 else palavras.map { it.length }.average()
    val palavraMaisLonga: String
```

```
        get() = palavras.maxByOrNull { it.length } ?: "Não disponível."
}
```

7) Porque é que `maxByOrNull` é preferido a `maxBy` em Kotlin? O que faz `?: "Não disponível"`?

R:

`maxBy` lança `NoSuchElementException` quando a lista está vazia, enquanto `maxByOrNull` devolve `null`. Assim, o caso vazio fica expresso no tipo (`String?`) e o compilador obriga a tratá-lo, em vez de falhar em tempo de execução. `?: "Não disponível."` substitui esse `null` por um valor por omissão, pelo que `palavraMaisLonga` é uma `String` não nula (Figura 16 (a)).

2.2.3. Funções e lambdas

O resumo passou para uma função de nível superior no ficheiro `Resumo.kt`, fora de qualquer classe. A lambda de filtragem foi guardada numa variável com o tipo explícito, e reaproveitou-se a lógica de `when` do passo anterior. A Figura 15 mostra o código e o resultado no Logcat, incluindo o `toString()` gerado pela `data class`.

```
const val LIMITE_PALAVRA_LONGA = 4

fun gerarResumo(bundle: PalavraBundle): String {
    val ehLonga: (String) -> Boolean = { it.length > LIMITE_PALAVRA_LONGA }
    val palavrasLongas = bundle.palavras.filter(ehLonga)
    val avaliacao = when (bundle.contar) {
        0 -> "Nenhuma palavra encontrada!"
        1 -> "Apenas uma palavra!"
        else -> "Foram digitadas ${bundle.contar} palavras."
    }
    return "$avaliacao\n" +
        "Comprimento médio: ${"%.2f".format(bundle.comprimentoMedio)}\n" +
        "Palavra mais longa: ${bundle.palavraMaisLonga}\n" +
        "Palavras longas (> $LIMITE_PALAVRA_LONGA letras): " +
        palavrasLongas.joinToString().ifEmpty { "nenhuma" }
}
```

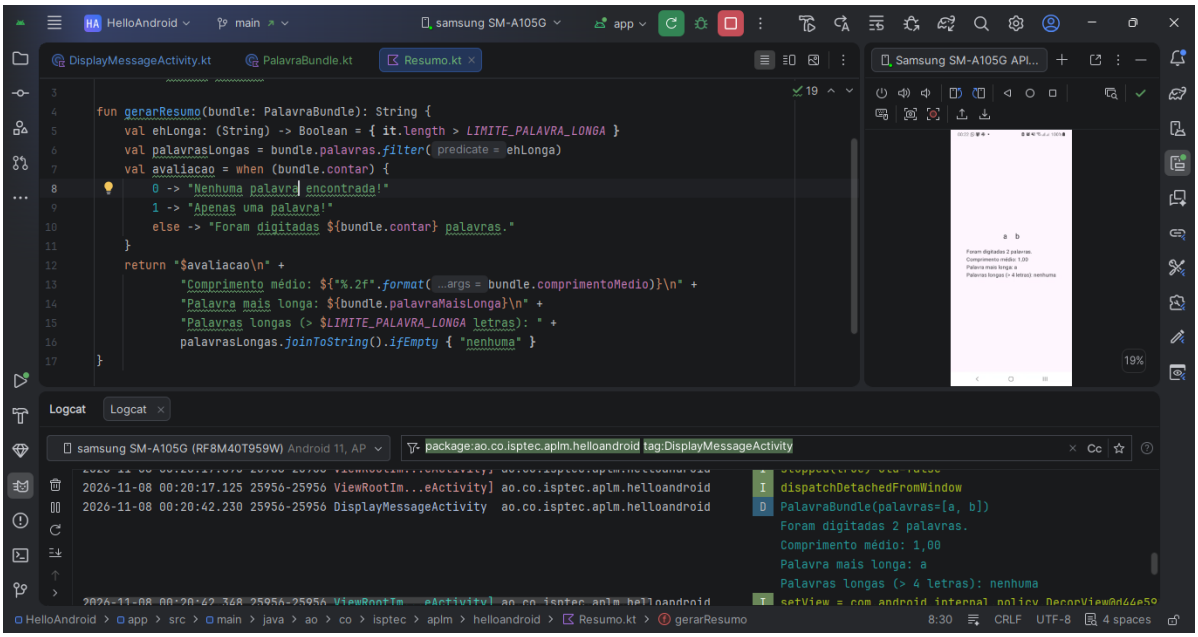


Figura 15 **–** Função gerarResumo em Resumo.kt e Logcat filtrado pela tag DisplayMessageActivity, com PalavraBundle(palavras=[a, b]) e o resumo gerado

8) Qual é o tipo da função lambda passada para `filter`? Qual a diferença entre esta e uma classe anónima em Java?

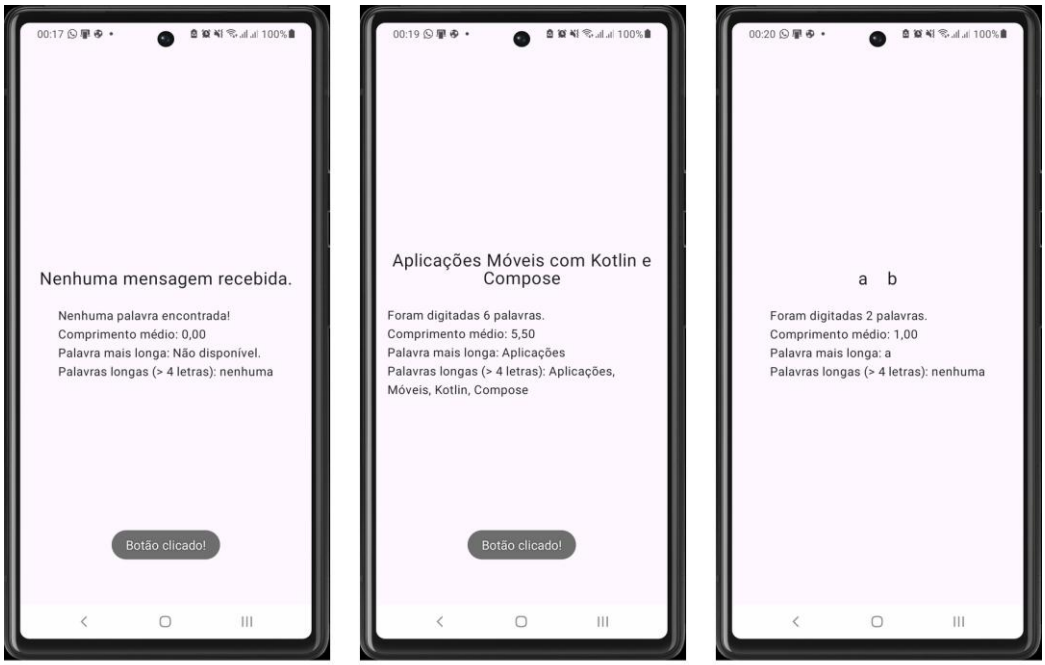
R:

O tipo é `(String) -> Boolean`, o do parâmetro `predicate` de `filter` para uma `List<String>`; no código está declarado explicitamente em `ehLonga`. Em Kotlin as funções são valores de primeira classe: a lambda é um literal conciso, com o parâmetro implícito `it`, e como `filter` é `inline` o seu corpo é inserido no local da chamada, sem criar um objecto. Uma classe anónima em Java exige declarar uma interface (por exemplo `Predicate<String>`) e o método a implementar, cria sempre uma instância, tem o seu próprio `this` e só pode usar variáveis locais *effectively final*.

2.2.4. Kotlin orientado a objectos e funcional

Por fim, a `DisplayMessageActivity` passou a construir um `PalavraBundle` a partir da mensagem e a apresentar o resumo num segundo `Text`, por baixo da mensagem original:

```
val mensagem = intent.getStringExtra(EXTRA_MESSAGE) ?: ""
val palavras = mensagem.split(" ").filter { it.isNotBlank() }
val bundle = PalavraBundle(palavras)
val resumo = gerarResumo(bundle)
Log.d(TAG, "$bundle\n$resumo")
```



(a) mensagem vazia (b) frase de 6 palavras (c) «a b»

Figura 16 – Resumo apresentado pela DisplayMessageActivity: (a) mensagem vazia, com média 0,00 e «Não disponível.»; (b) frase de seis palavras; (c) duas palavras separadas por vários espaços

9) Onde armazenaria classes auxiliares como `PalavraBundle` num grande projecto Android?

R:

Segundo o guia de arquitectura, o código organiza-se por camadas: `ui` (actividades, composables e ViewModels), `domain` (regras de negócio e casos de uso), `data` (modelos, repositórios e fontes de dados) e `util` (funções genéricas e de extensão). Aqui, `PalavraBundle` seria um modelo em `data.model` (ou `domain.model`), `gerarResumo` um caso

de uso em `domain` e a `DisplayMessageActivity` ficaria em `ui`. Em projectos maiores, estes pacotes repetem-se por funcionalidade ou separam-se em módulos Gradle. Neste laboratório, pela sua dimensão, ficaram todos no pacote principal.

2.3. Exercício III - Debugging e Logcat

Este exercício explora as ferramentas de depuração do Android Studio sobre o projecto HelloAndroid.

2.3.1. Filtros do Logcat

Acrescentaram-se mensagens de vários níveis: `Log.i` em `onCreate()` e nos restantes métodos do ciclo de vida (`onStart`, `onResume`, `onPause`, `onStop`, `onDestroy`), `Log.w` quando se envia uma mensagem vazia e `Log.e` para um erro intencional. Para este último criou-se um botão «Provocar erro (÷ 0)» que chama uma função de divisão inteira por zero, o que lança `ArithmeticException`; a excepção é apanhada e registada com a pilha de chamadas:

```
fun dividir(a: Int, b: Int): Int = a / b

try {
    val resultado = dividir(10, 0)
    Log.d(TAG, "Resultado: $resultado")
} catch (e: ArithmeticException) {
    Log.e(TAG, "Erro intencional: divisão por zero", e)
}
```

No Logcat combinaram-se, numa só consulta, filtros por tag (`tag:MainActivity`, `tag:AndroidRuntime`), por pacote (`package:mine`) e por nível (`level:info`, `level:warn`, `level:error`). A Figura 17 mostra os métodos do ciclo de vida e a entrada de nível E com a excepção `java.lang.ArithmeticException: divide by zero`, cuja pilha aponta para `MainActivityKt.dividir`.

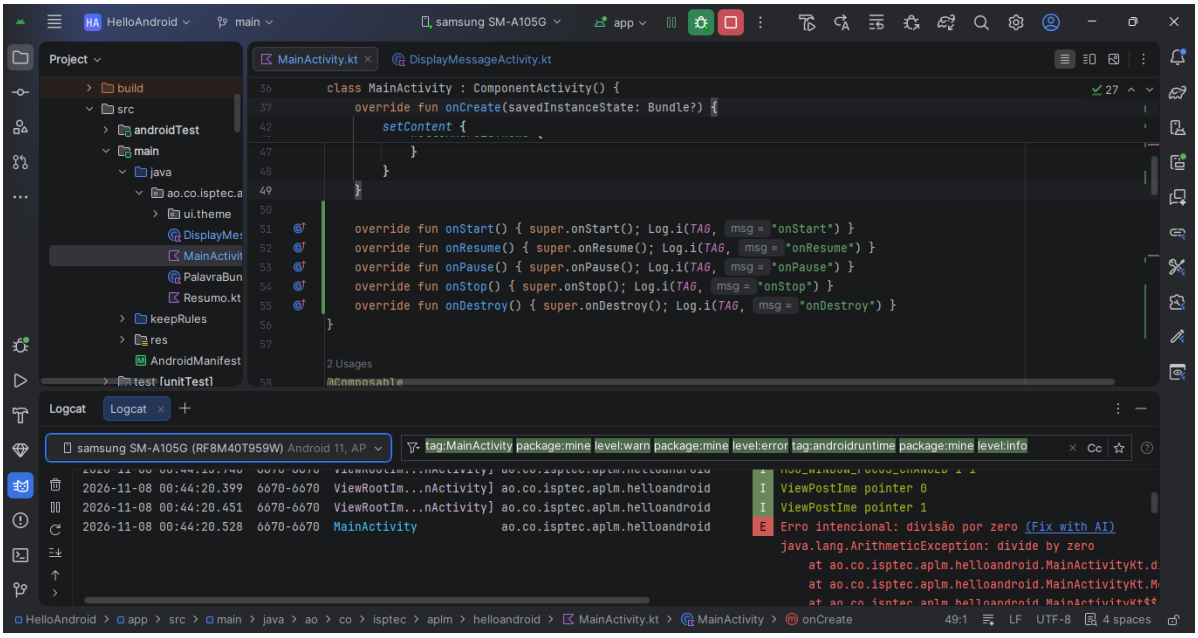


Figura 17 **–** Logs do ciclo de vida (Log.i) e erro intencional registado com Log.e: ArithmeticException «divide by zero» e respectiva pilha no Logcat filtrado

2.3.2. Breakpoints e o debugger

Colocou-se um breakpoint na linha do `Log.i` de `onCreate()` e a aplicação foi lançada em modo de depuração (ícone do insecto). Com a execução suspensa, o editor mostra os valores em linha (`savedInstanceState: null`, por ser o primeiro arranque). Um **Step Over** executou essa linha e avançou para `enableEdgeToEdge()`; em consequência, a mensagem «onCreate (savedInstanceState = null)» surge no Logcat (Figura 18).

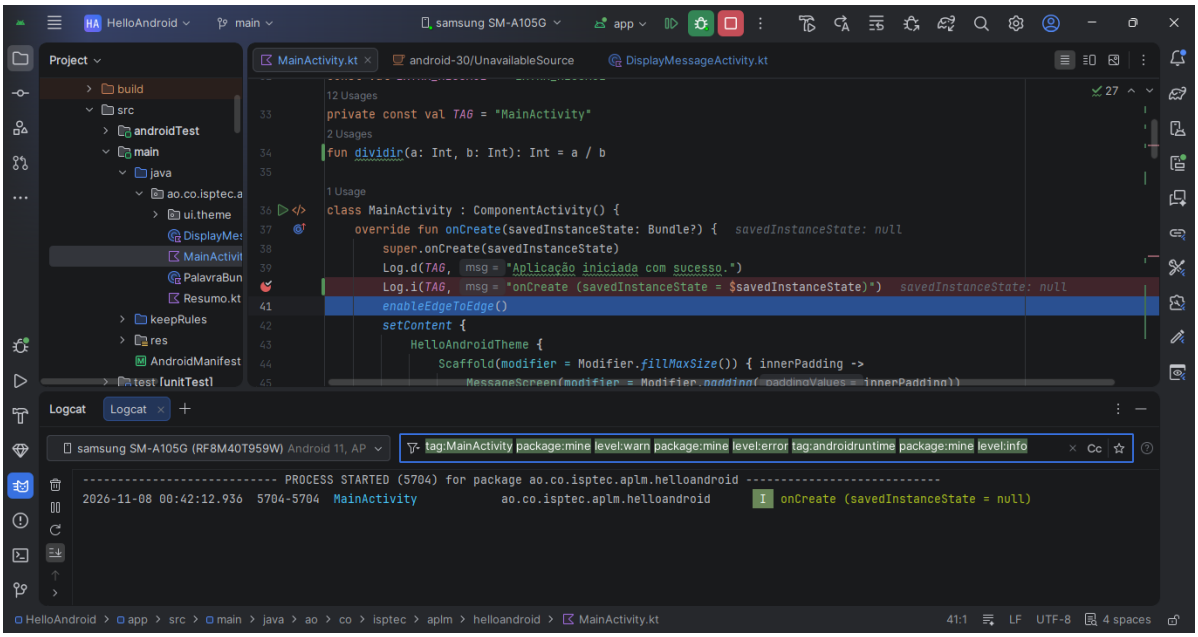


Figura 18 – Breakpoint no Log.i de onCreate(): após Step Over, a execução está na linha seguinte e a mensagem já surge no Logcat

A janela **Debug** (Figura 19) apresenta, no separador *Threads & Variables*, a thread seleccionada (`main`), a pilha de chamadas e as variáveis do frame actual. A pilha mostra que `onCreate` é invocado pela framework: `MainActivity.onCreate` ← `Activity.performCreate` ← `Instrumentation.callActivityOnCreate` ← `ActivityThread.performLaunchActivity`. Na barra da janela encontram-se os comandos Step Over, Step Into, Step Out e Resume Program.

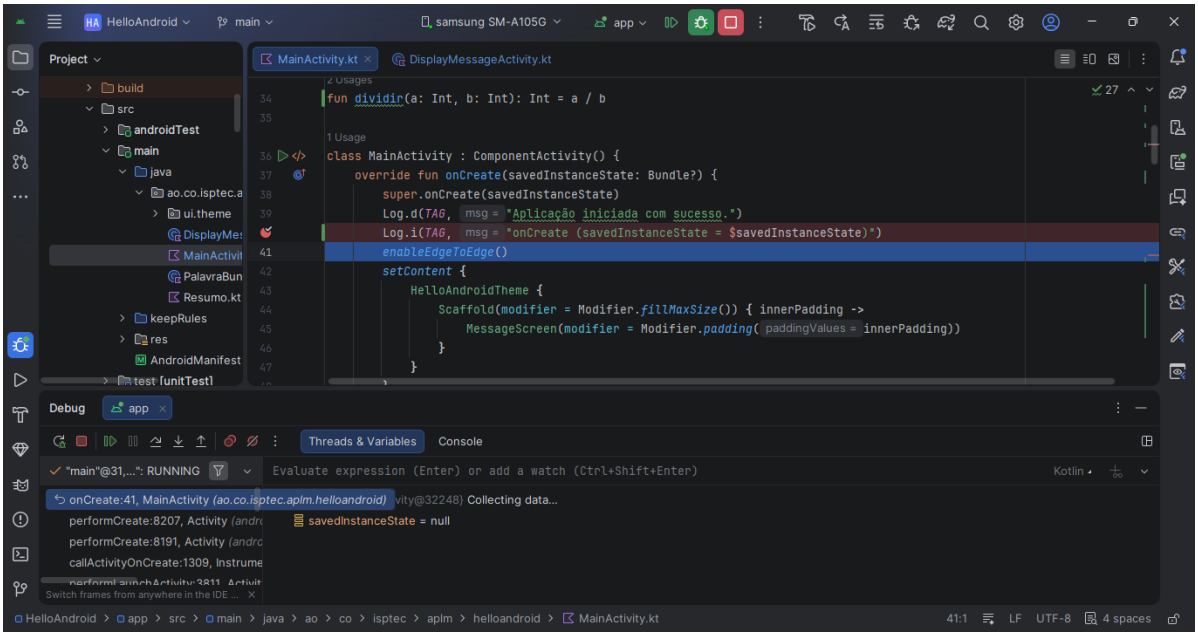


Figura 19 **–** Janela Debug: thread main, pilha de chamadas até MainActivity.onCreate e variável savedInstanceState = null

10) O que acontece quando inspecciona um objecto no painel Variáveis? O que fazem as opções "Step Over" e "Step Into"?

R:

O painel mostra as variáveis visíveis no frame seleccionado com o respectivo valor; ao expandir um objecto, apresentam-se os seus campos (incluindo os privados e herdados) com o tipo e o identificador da instância, de forma recursiva, e os nulos aparecem como `null`. A execução mantém-se suspensa, e é possível alterar valores (Set Value), acompanhá-los em Watches ou avaliar expressões (Evaluate Expression). **Step Over** (F8) executa a linha actual por inteiro, incluindo as funções que chama, e pára na linha seguinte do mesmo método. **Step Into** (F7) entra na função chamada nessa linha e pára na sua primeira instrução; **Step Out** regressa ao chamador.

2.3.3. Inspecção do estado de execução

Com a execução suspensa num breakpoint, o selector de threads da janela Debug (Figura 19) permite passar da thread `main`, onde corre a interface, para as restantes threads do processo (por exemplo `RenderThread` e as threads `binder`) e ver a pilha de cada uma. Num breakpoint dentro do `onClick` do botão «Enviar», o estado `text` surge no painel Variables como `text$delegate`, o `MutableState` criado por `remember`, cujo valor é a mensagem escrita. O uso de memória da aplicação analisa-se na janela **Profiler** (View > Tool Windows > Profiler), separada do debugger.

2.4. Exercício IV - Aplicação Lista de Tarefas

O último exercício reúne o que foi visto numa aplicação de lista de tarefas com uma única actividade. Criou-se um novo projecto com o modelo Empty Activity (Figura 20): nome *Lista de Tarefas*, pacote `ao.co.isptec.aplm.listadetarefas`, guardado em `Laboratorio02/Parte_2/ListadeTarefas`, com o SDK mínimo API 24.

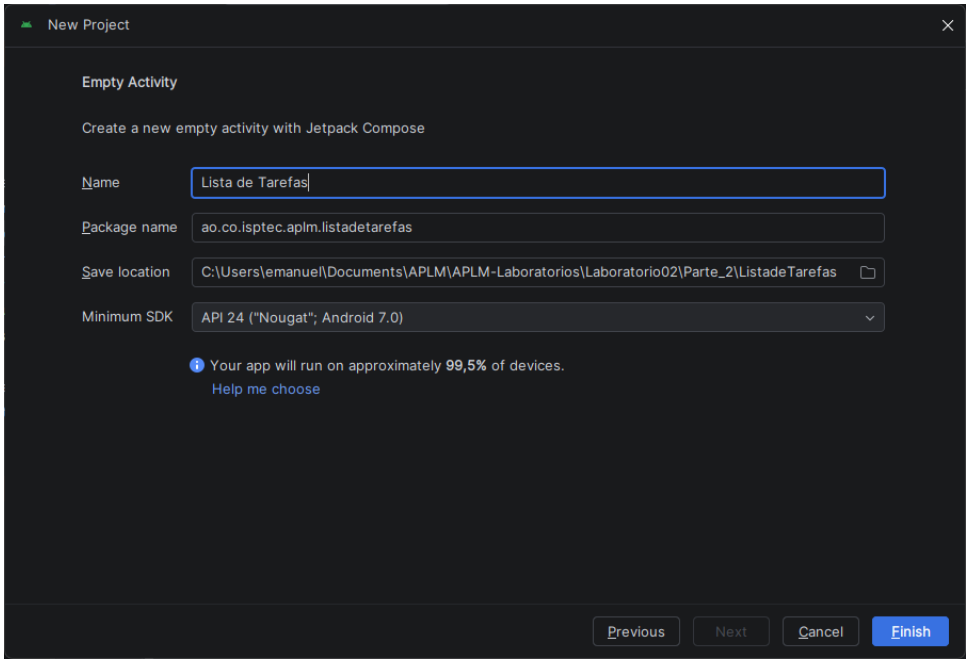


Figura 20 **–** Configuração do projecto Lista de Tarefas: pacote ao.co.isptec.aplm.listadetarefas e SDK mínimo API 24

2.4.1. Estrutura da interface do utilizador

A primeira versão seguiu o código do enunciado: o composable `TaskScreen` contém uma `Column` com um `OutlinedTextField`, um botão «Add Task» e uma `LazyColumn`, e as tarefas são guardadas como `String` numa `mutableStateListOf<String>()`. O botão ignora textos em branco, acrescenta a tarefa e limpa o campo:

```
val taskList = remember { mutableStateListOf<String>() }
var text by remember { mutableStateOf("") }

Button(onClick = {
    if (text.isNotBlank()) {
        taskList.add(text)
        text = ""
    }
}) { Text("Add Task") }

LazyColumn {
    items(taskList) { task -> Text(task, modifier = Modifier.padding(8.dp)) }
}
```

2.4.2. Modelo de dado Task

As tarefas passaram a ser objectos da classe de dados `Task`, numa `mutableStateListOf<Task>()`. Cada linha é desenhada pelo composable `TaskRow`, com uma `Checkbox` e o título riscado (`TextDecoration.LineThrough`) quando a tarefa está concluída. Como `Task` é imutável, a conclusão substitui o elemento da lista por uma cópia:

```
data class Task(val title: String, val isDone: Boolean = false)

fun alternarConclusao(task: Task) {
    val i = tasks.indexOf(task)
    if (i >= 0) tasks[i] = task.copy(isDone = !task.isDone)
}
```

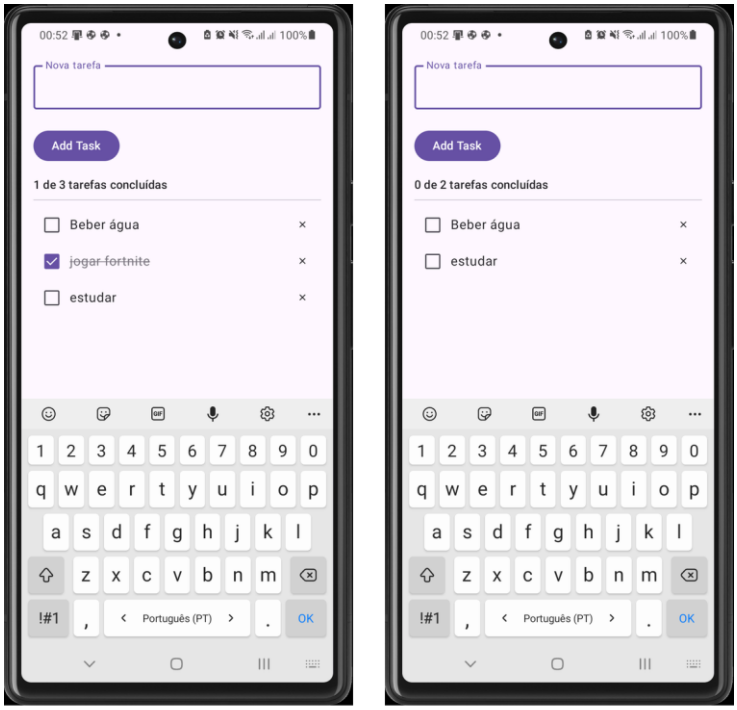
2.4.3. Gerir eventos

A adição foi concentrada na função `adicionarTarefa()`, chamada tanto pelo botão «Add Task» como pelo botão OK do teclado (`KeyboardActions(onDone = …)`). A função lê e normaliza a entrada com `trim()`, rejeita os campos vazios e os duplicados (sem distinguir maiúsculas de minúsculas), acrescenta a `Task` e limpa o campo. O erro aparece por baixo do campo, através de `isError` e `supportingText`, e desaparece quando o utilizador volta a escrever:

```
fun adicionarTarefa() {
    val titulo = text.trim()
    erro = when {
        titulo.isEmpty() -> "Escreva uma tarefa."
        tasks.any { it.title.equals(titulo, ignoreCase = true) } ->
            "Essa tarefa já existe."
        else -> null
    }
    if (erro == null) {
        tasks.add(Task(titulo))
        Log.d(TAG, "Tarefa adicionada: $titulo (total = ${tasks.size})")
        text = ""
    } else {
        Log.w(TAG, "Tarefa rejeitada: $erro")
    }
}
```

2.4.4. Recursos avançados

Foram implementados os dois recursos opcionais. A conclusão alterna-se tocando na linha ou na caixa de selecção. Para apagar, cada linha tem um botão «×» (`IconButton`); preferiu-se este botão ao «toque para remover» porque o toque na linha já alterna a conclusão. Acrescentou-se ainda o contador «X de Y tarefas concluídas». A Figura 21 mostra a aplicação no dispositivo.



(a) 3 tarefas, 1 concluída (b) após apagar a concluída

Figura 21 **–** Lista de Tarefas no dispositivo: (a) três tarefas, uma marcada como concluída (riscada) e contador «1 de 3»; (b) após apagar a tarefa concluída com «×», o contador passa a «0 de 2»

3. Desafios

No laboratório conceberam-se duas aplicações Android em Kotlin e Jetpack Compose, ambas organizadas num único módulo Gradle (`app`) e num único pacote. A **HelloAndroid** tem duas actividades: a primeira recolhe uma mensagem e a segunda apresenta-a com estatísticas sobre as palavras. A **Lista de Tarefas** tem uma única actividade, que gere uma lista de tarefas em memória. A Tabela 1 resume a estrutura do software.

Tabela 1 **–** Estrutura do software das duas aplicações

Aplicação	Ficheiro	Conteúdo e responsabilidade
HelloAndroid	`MainActivity.kt`	`MainActivity` (actividade inicial), composable `MessageScreen`, constante `EXTRA_MESSAGE` e função `dividir`
	`DisplayMessageActivity.kt`	segunda actividade: lê o extra da Intent e apresenta a mensagem e o resumo
	`PalavraBundle.kt`	`data class` com as estatísticas da lista de palavras
	`Resumo.kt`	função de nível superior `gerarResumo` e constante `LIMITE_PALAVRA_LONGA`
Lista de Tarefas	`MainActivity.kt`	`MainActivity`, composables `TaskScreen` (estado e eventos) e `TaskRow` (uma linha)
	`Task.kt`	`data class Task`, o modelo de cada tarefa
Ambas	`ui/theme/`	tema Material 3 gerado pelo modelo (`Color.kt`, `Theme.kt`, `Type.kt`)
	`AndroidManifest.xml`	declaração das actividades; só a `MainActivity` tem o `intent-filter` LAUNCHER

3.1. Desenho da interface móvel

Na **HelloAndroid** (Figura 22), a `MainActivity` mostra uma caixa de texto e os botões «Enviar» e «Provocar erro (÷ 0)». «Enviar» dá feedback com um Toast e abre a `DisplayMessageActivity` através de uma Intent explícita que leva a mensagem como extra. A segunda actividade apresenta a mensagem e, por baixo, o resumo gerado a partir dela. O botão Back fecha-a e regressa à primeira.

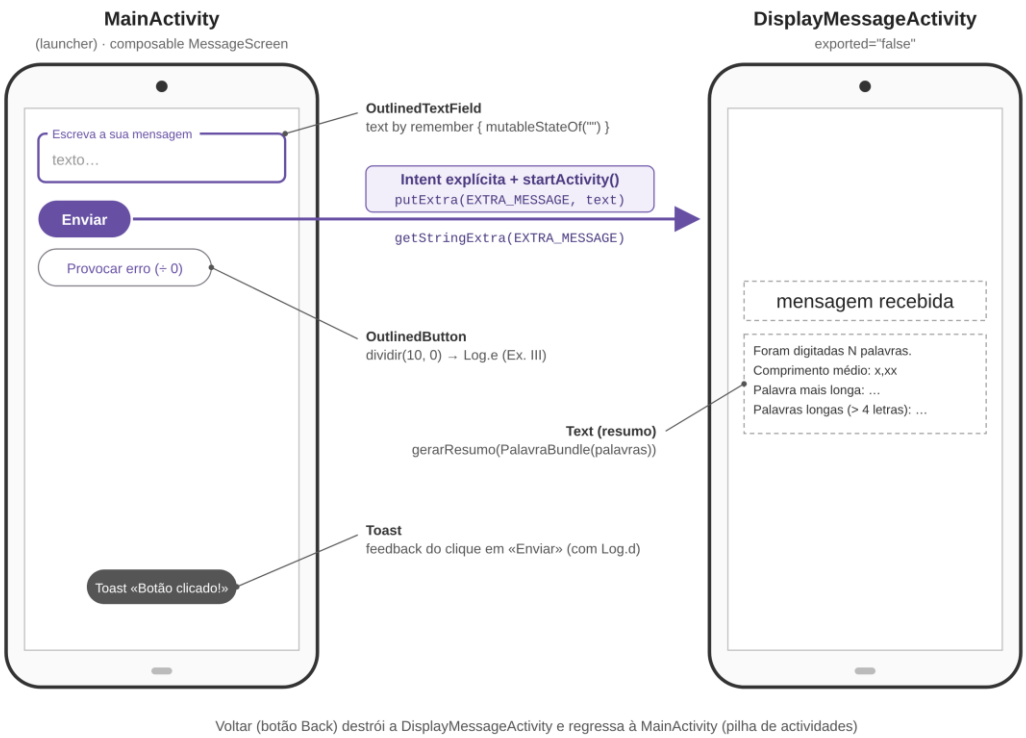


Figura 22 **–** Wireframe das actividades da aplicação HelloAndroid e navegação entre elas através de uma Intent explícita

Na **Lista de Tarefas** (Figura 23), a única actividade dispõe os elementos na vertical: o campo «Nova tarefa», com espaço para a mensagem de erro; o botão «Add Task»; o contador de tarefas concluídas; e a `LazyColumn`, que ocupa o resto do ecrã. Cada linha tem uma caixa de selecção, o título e o botão «⨯».

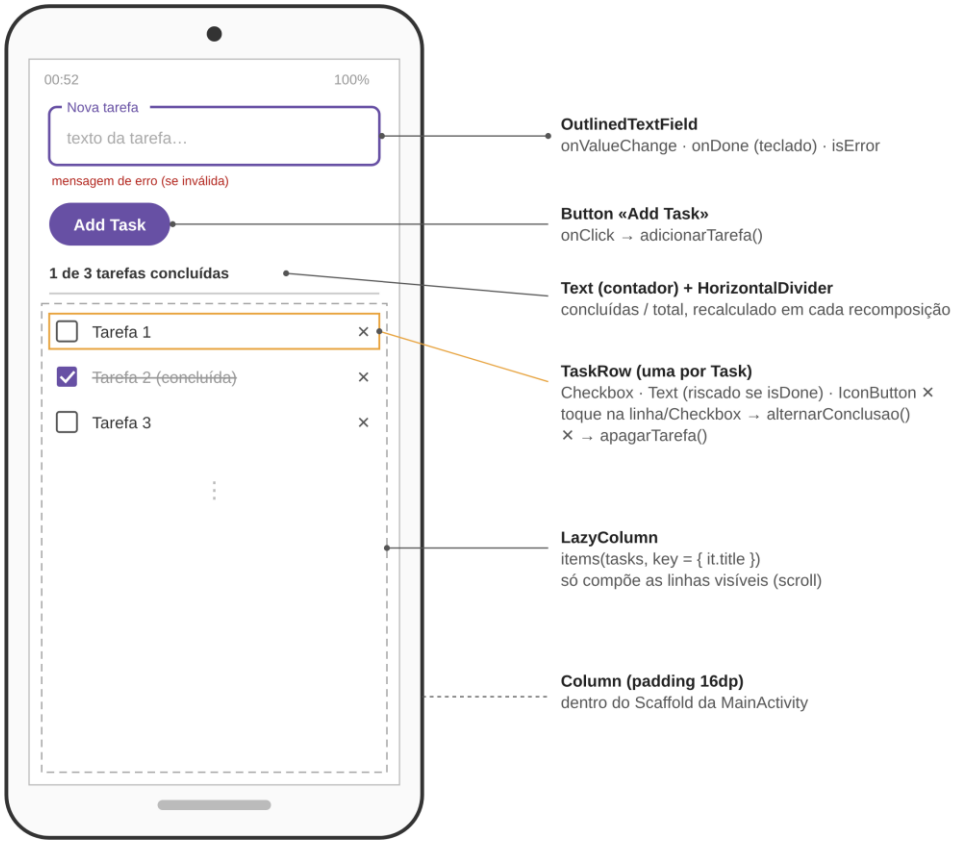


Figura 23 **–** Wireframe da actividade da aplicação Lista de Tarefas, com os composables e os eventos associados a cada elemento

3.2. Detalhes de implementação

Arquitectura. As duas aplicações seguem o modelo do Jetpack Compose: cada actividade estende `ComponentActivity` e limita-se a chamar `setContent { }` com o tema; a interface é composta por funções `@Composable`, redesenhadas sempre que o estado que lêem muda. O fluxo de dados é unidireccional: o estado desce para os composables como parâmetros e os eventos sobem como callbacks. Na Lista de Tarefas, `TaskScreen` guarda o estado e define as funções que o alteram; `TaskRow` não tem estado próprio e recebe a tarefa e as funções `onToggle` e `onDelete`. Na HelloAndroid, a navegação usa a pilha de actividades do sistema, e o processamento da mensagem (`PalavraBundle` e `gerarResumo`) é independente da interface, pelo que poderia passar para um pacote `domain` numa aplicação maior.

Estruturas de dados. Na HelloAndroid, a mensagem é um `MutableState<String>` criado com `remember`, transportada entre actividades como extra da Intent (um par chave-valor, com a chave `EXTRA_MESSAGE`). No destino é dividida numa `List<String>`, sem os elementos vazios, e encapsulada no `PalavraBundle`, cujas estatísticas são propriedades calculadas. Na Lista de Tarefas, cada tarefa é uma `Task` imutável e as tarefas ficam numa `SnapshotStateList` (`mutableStateListOf<Task>()`), uma lista observável: inserções, remoções e substituições provocam a recomposição sem chamadas de actualização explícitas (ao contrário do `notifyDataSetChanged()` da Parte 1). O texto do campo usa `rememberSaveable`, a mensagem de erro é um `String?`, e o contador é um valor derivado (`tasks.count { it.isDone }`). Na `LazyColumn`, cada linha é identificada pelo título (`key = { it.title }`), único porque não se aceitam duplicados.

```
data class Task(val title: String, val isDone: Boolean = false)

val tasks = remember { mutableStateListOf<Task>() }        // lista observável de Task
var text by rememberSaveable { mutableStateOf("") }        // texto do campo
var erro by remember { mutableStateOf<String?>(null) }     // mensagem de validação
```

Eventos. Na HelloAndroid, os eventos são a escrita no campo (`onValueChange`) e os cliques nos dois botões (`onClick`). A Tabela 2 resume os eventos da Lista de Tarefas: nenhum altera a interface directamente; alteram o estado, e a interface é redesenhada a partir dele.

Tabela 2 **–** Eventos tratados pela aplicação Lista de Tarefas

Evento	Componente · callback	Tratamento	Efeito no estado e na UI
Escrita no campo	`OutlinedTextField` · `onValueChange`	`text = it`; `erro = null`	campo actualizado; erro limpo
Clique em «Add Task»	`Button` · `onClick`	`adicionarTarefa()`	valida; adiciona a `Task` e limpa o campo, ou mostra o erro
OK do teclado virtual	`KeyboardActions` · `onDone`	`adicionarTarefa()`	igual ao botão, sem tirar o teclado
Toque na linha ou na caixa	`Row.clickable` / `Checkbox` · `onCheckedChange`	`alternarConclusao(task)`	`tasks[i] = task.copy(isDone = !isDone)`; título riscado; contador
Clique em «X»	`IconButton` · `onClick`	`apagarTarefa(task)`	`tasks.remove(task)`; linha removida; contador
Deslizar a lista	`LazyColumn` (scroll)	interno ao Compose	compõe apenas as linhas que entram no ecrã

Comportamento. A Figura 24 mostra o tratamento de cada evento da Lista de Tarefas e a recomposição que dele resulta. As acções são registadas no Logcat com a tag `ListaDeTarefas`, e as da HelloAndroid com as tags `MainActivity` e `DisplayMessageActivity`.

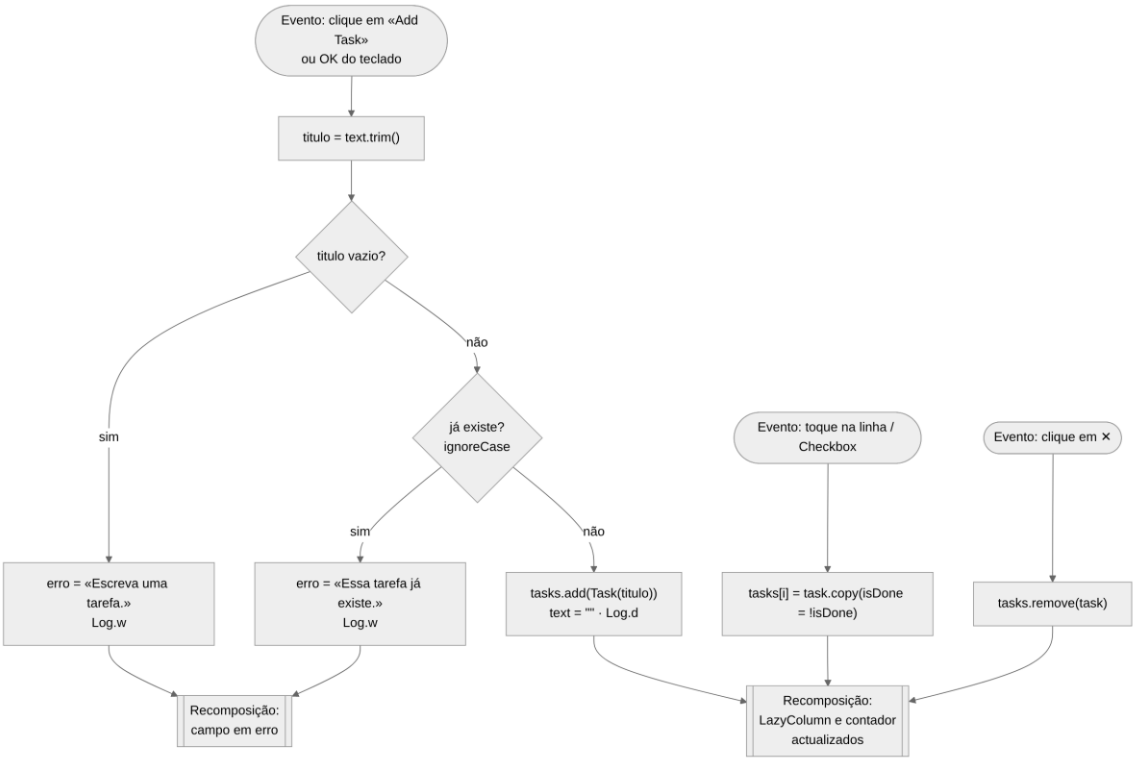


Figura 24 **–** Comportamento da aplicação Lista de Tarefas: tratamento dos eventos de adição, conclusão e remoção, e recomposição resultante

Outros detalhes técnicos. Usou-se o texto «⨯» em vez de `Icons.Default.Delete` porque as versões recentes do Material 3 já não incluem a biblioteca de ícones. Como limitação, os dados das duas aplicações existem apenas em memória: na Lista de Tarefas, o texto do campo sobrevive a uma rotação (`rememberSaveable`), mas a lista perde-se quando a actividade é recriada ou a aplicação fechada. Resolver-se-ia com um `ViewModel` e persistência local (DataStore ou Room).

4. Referências Bibliográficas

Android Developers. (s.d.). *Create and manage virtual devices*. https://developer.android.com/studio/run/managing-avds

Android Developers. (s.d.). *Create your first Android app*. https://developer.android.com/codelabs/basic-android-kotlin-compose-first-app

Android Developers. (s.d.). *Debug your app*. https://developer.android.com/studio/debug

Android Developers. (s.d.). *Guide to app architecture*. https://developer.android.com/topic/architecture

Android Developers. (s.d.). *Introduction to activities*. https://developer.android.com/guide/components/activities/intro-activities

Android Developers. (s.d.). *Kotlin style guide*. https://developer.android.com/kotlin/style-guide

Android Developers. (s.d.). *Learn the Kotlin programming language*. https://developer.android.com/kotlin/learn

Android Developers. (s.d.). *Lists and grids*. https://developer.android.com/develop/ui/compose/lists

Android Developers. (s.d.). *Projects overview*. https://developer.android.com/studio/projects

Android Developers. (s.d.). *Run apps on a hardware device*. https://developer.android.com/studio/run/device

Android Developers. (s.d.). *State and Jetpack Compose*. https://developer.android.com/develop/ui/compose/state

Android Developers. (s.d.). *View logs with Logcat*. https://developer.android.com/studio/debug/logcat

Costa, J. (2026). *Laboratório #02 – Parte 2: Introdução ao Android e ao Kotlin* [Enunciado de laboratório, Aplicações Móveis, ano lectivo 2026-27]. Instituto Superior Politécnico de Tecnologias e Ciências (ISPTEC).

JetBrains. (s.d.). *Data classes*. Kotlin Documentation. https://kotlinlang.org/docs/data-classes.html

JetBrains. (s.d.). *Higher-order functions and lambdas*. Kotlin Documentation. https://kotlinlang.org/docs/lambdas.html

JetBrains. (s.d.). *Kotlin language reference*. https://kotlinlang.org/docs/reference/

JetBrains. (s.d.). *Null safety*. Kotlin Documentation. https://kotlinlang.org/docs/null-safety.html

5. Repositório GitHub

Repositório: github.com/SW-Wanted/APLM-Laboratorios.[1]

[1] Foi enviado um convite de colaborador para o utilizador **joaojdacosta**.